

FlowOps — Architecture & Design

Brian Duffy
June 9, 2026

CONTENTS

title: "FlowOps — Architecture & Design Document" subtitle: "Governed self-service provisioning: the governed actuation layer" author: "Brian Duffy" date:

FlowOps — Architecture & Design Document

1. Executive Summary	
2. Objectives	
2.1 Positioning	
2.2 Landscape	
3. Scope	
3.1 In scope (v1)	
3.2 Explicitly NOT in scope (v1)	
3.3 Honest tension	
4. Architecture	
4.1 System overview	
4.2 Execution model — durable queue + serverless container job	
4.3 Isolation — account/project per sandbox	
4.4 The Actuator seam (the only abstraction in v1)	
4.5 Budget gate — cloud-enforced, not declared	
4.6 Domain model	
4.7 Execution profiles — dev (local) mode	
5. Engineering Considerations	
5.1 Failure modes	
5.2 Testing strategy	
5.3 Security & supply chain	
5.4 Tooling choices (boring by default)	
5.5 Distribution	
6. Design	
6.1 Design system — NYS Design System (mandatory)	
6.2 Information architecture — sandbox lifecycle view	
6.3 Approved mockup	
6.4 Interaction-state coverage	

7. Strategy	
8. Build Plan	
8.1 Parallel lanes	
8.2 Sequenced tasks (highlights)	
8.5 Security Requirements (CSO threat model, 2026-06-09)	
9. Open Risks	
Appendix A — Review Provenance	

title: “FlowOps — Architecture & Design Document” subtitle: “Governed self-service provisioning: the governed actuation layer” author: “Brian Duffy” date: “June 8, 2026”

FlowOps — Architecture & Design Document

Status: Plan approved (Eng + CEO + Design reviews cleared). Pre-implementation. **Version:** 1.0 (v1 scope) **Document owner:** Brian Duffy

1. Executive Summary

FlowOps is an open-source **governed self-service provisioning platform** — *the governed actuation layer*. It lets a person request infrastructure, pass a policy and budget gate, and have it actually **built** — with controls (approval, budget, policy, audit) that never come off, and with agents and automation able to do the work *inside* the guardrails.

It begins as a refined work-intake Kanban proof of concept and grows into something sharper: **AI-native ITSM**. ITSM's skeleton — `request → task → action`, cleanly linked — is genuinely good. Its flesh — the human bureaucracy that fulfills every task — is pure toil. ServiceNow won in 2005 because Remedy made you waste your life on care-and-feeding. In 2026 the toil worth killing is the *human-in-every-step* fulfillment. FlowOps keeps the elegant `request → task → action` model and makes a task's fulfiller a human, an automation, or an AI agent — while governance stays constant no matter who or what does the work.

The v1 wedge is **sandbox vending**: a developer (or AI) sandbox that is requested, gated, provisioned, and auto-torn-down on a TTL — proving the whole governed-actuation loop end to end. It lives in the unserved middle between ClickOps (manual, ungoverned, doesn't scale) and full CI/CD pipelines (overkill, get in the way).

This document captures the objectives, the chosen approach, the engineering architecture, the design system, and the strategy, as decided across four structured reviews (product, engineering, strategy, design).

2. Objectives

1. **Kill one unit of toil for real.** A non-author colleague requests a dev sandbox, it passes an approval + cloud-enforced budget gate, real infrastructure is provisioned and auto-torn-down on TTL — with a full audit trail — without anyone hand-running infrastructure-as-code.
2. **Make governance first-class, not bolted on.** Every fulfilled task is gated and audited. The controls are the product.
3. **Be genuinely open source and embeddable.** A single organization can run the complete governed-actuation loop for free; the platform is designed to be embedded in other internal platforms.
4. **Earn the right to the bigger vision.** Prove one concrete loop, then generalize to the agentic workflow engine and the embeddable actuation core.

2.1 POSITIONING

The category FlowOps claims is "**governed self-service provisioning — the governed actuation layer.**" The headline is governance + actuation as one loop ("a request passes a gate and gets *built*, and the controls never come off"). The sandbox is the concrete demo *under* the category; "AI-native ITSM" is the vision/roadmap narrative.

This matters because category sets the comparison set. Called a "sandbox tool," FlowOps is a feature next to Backstage's scaffolder. Called "the governed actuation layer," it defines a lane it can own.

2.2 LANDSCAPE

Space	Examples	Why FlowOps is different
OSS ITSM	GLPI, Zammad, iTop, OTOBO, osTicket	Those track tickets. FlowOps <i>actuates</i> and governs.
IDP / platform	Backstage, Port, Cortex, Humanitec	Those are catalogs + scaffolding, often BYO-everything. FlowOps is a batteries-included governed loop .
Actuation	Crossplane, AWS Service Catalog, Spacelift, env0	Those provision. FlowOps provisions <i>inside</i> gates with audit, as one loop.
Commercial ITSM	ServiceNow, Atlassian JSM	FlowOps is the AI-native, open-source reframe.

The seam almost nobody owns is the *middle*: intake + governance + actuation as one loop, where the gate, the budget check, the policy enforcement, and the provisioning are the same system, and agents do the labor inside the guardrails.

3. Scope

3.1 IN SCOPE (V1)

The minimal loop that proves the thesis:

`request → task → action` , with one blueprint (**dev sandbox**), one gate set (approval

- cloud-enforced budget), one real actuator (OpenTofu in a container job), TTL teardown, and an append-only audit trail. Plus a **dev (local) execution profile** that runs the entire loop on a laptop with zero cloud credentials (see §4.7).

3.2 EXPLICITLY NOT IN SCOPE (V1)

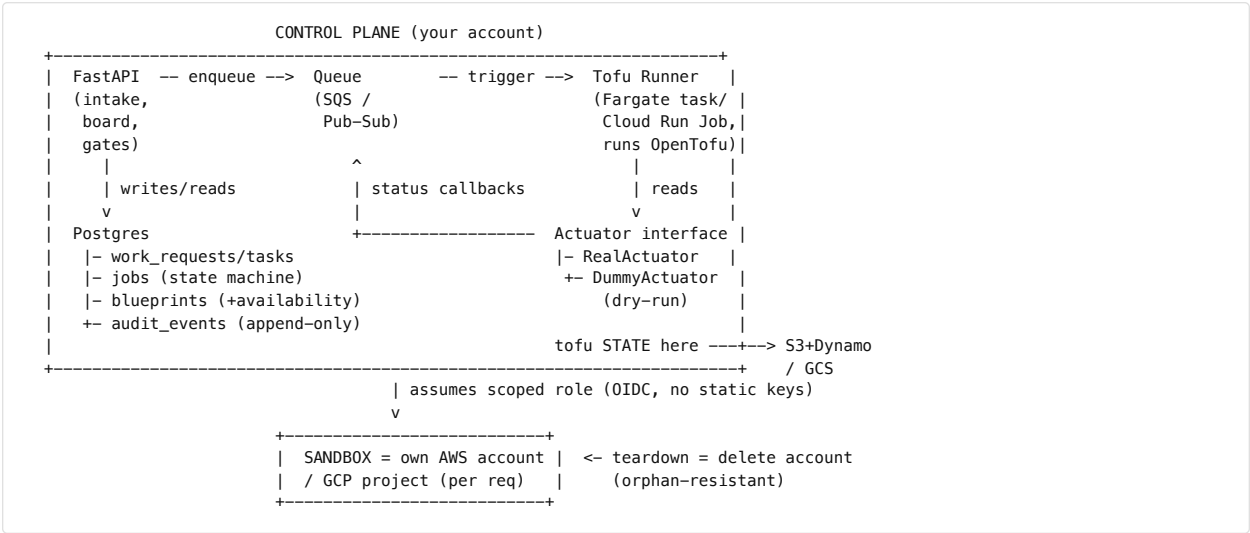
Deferred	Rationale
AI sandbox blueprint	Fast-follow (#2 blueprint); proves the actuator seam. Needs definition first.
Agent fulfiller	The emotional center of the project, deliberately deferred to protect credibility — v1 proves the <i>loop and gates</i> , not yet the thesis.
Plugin registry / Gate & Fulfiller interfaces	Extract on the second real instance, not before.
Live cost estimation (Infracost)	Cloud-enforced budget replaces it for v1.
Warm pool of pre-vended accounts	Perf optimization for when on-demand vending latency hurts.
Multi-tenancy, SSO, hosted runners	Commercial / open-core layer, post-v1.
LocalStack dev path (real OpenTofu vs emulated AWS)	Fast-follow to dev mode; v1 dev mode is Dummy-only (§4.7).
<code>local-docker</code> blueprint (usable local sandbox)	Fast-follow; lets dev mode build a real local container, not just a simulated handle.

3.3 HONEST TENSION

The most exciting part of the vision — agents doing the work inside the gates — is *not* in v1. v1 proves the governance + actuation loop with human-approval + automation only. This is the right call: it keeps the “it actually gets built” promise honest. The agent fulfiller is fast-follow #1.

4. Architecture

4.1 SYSTEM OVERVIEW



4.2 EXECUTION MODEL — DURABLE QUEUE + SERVERLESS CONTAINER JOB

`tofu apply` / `destroy` is a multi-minute operation that can fail halfway. A synchronous HTTP handler cannot own it. FlowOps uses a **durable queue (SQS / Pub-Sub) feeding a serverless container job** (AWS Fargate task / GCP Cloud Run Job) that runs OpenTofu.

- No 15-minute function ceiling; crash-safe via queue redelivery.
- A `jobs` table is the state machine. **One job path serves both apply and destroy.**
- Required mechanics: idempotency keys, per-sandbox locks, job leases, and “apply-succeeded-but-callback-failed” reconciliation.
- `status()` reads the jobs table — it is not a live cloud poll.

```
JOB STATE MACHINE (one path for apply AND destroy):
  queued -> planning -> awaiting_gate -> applying -> succeeded
      |           |           |           |           |
      v           | (deny)    |           |           |
      failed      v          failed<--+   v (ttl)    destroying -> destroyed
                        +--- reconcile / orphan-sweep
```

4.3 ISOLATION — ACCOUNT/PROJECT PER SANDBOX

Each sandbox is its **own AWS account** (Organizations) or **GCP project**. The runner assumes a scoped role via OIDC — no stored static keys.

- **Teardown = delete the account/project.** This makes the two scariest problems structurally safe: a failed apply or a missed orphan is recovered by deleting the whole account/project, not by hoping a tag-sweep caught everything.
- Orphan-*resistant*, not instant or perfect: AWS account closure has a ~90-day window, GCP project deletion a ~30-day recovery window, plus billing tails — design for the tail.
- OpenTofu **state lives in the control-plane account**, encrypted, with secrets redacted and access isolated.

The first cloud (AWS vs GCP) is decided in a **week-0 spike** that builds the apply/destroy/orphan test against both and picks the factory that is demonstrably safe and fast (GCP project vending is roughly one API call; AWS account factory is heavier OU/SCP/Control Tower ceremony).

4.4 THE ACTUATOR SEAM (THE ONLY ABSTRACTION IN V1)

```
class Actuator(Protocol):
    def plan(self, intent: Intent) -> Preview: ...      # pre-flight: what + est. cost
    def apply(self, intent: Intent) -> Handle: ...      # returns state-locating handle
    def destroy(self, handle: Handle) -> Result: ...
    def status(self, handle: Handle) -> Status: ...
```

Two v1 implementations: `RealActuator` (the container job) and `DummyActuator` (dry-run). The dummy is also the test double *and* the placeholder for not-yet-live capabilities — it lets the whole loop be built, demoed, and tested before the cloud is wired. Capability flags (`available` | `preview` | `dry-run-only`) let the UI build around absent features, but absent capabilities are shown as absent/preview, never silently faked.

Gates (approval, budget) and `fulfiller_type` stay concrete — no plugin registry until a real second instance forces the interface shape.

4.5 BUDGET GATE — CLOUD-ENFORCED, NOT DECLARED

The budget gate is a **hard ceiling the cloud enforces**, not a user promise. At provision time, FlowOps sets a real AWS Budget + SCP / GCP budget + quota + service allowlist on the sandbox account. A declared “this fits under \$X” is governance theater; the cloud-enforced ceiling is the only real control — and it removes the need for live cost estimation in v1.

4.6 DOMAIN MODEL

`request -> task -> action`, kept thin (no ITSM ceremony — no SLAs, categories, queues). The task *is* the provisioning job; the action *is* the actuator call. This keeps the thesis structurally true so the agent-fulfiller future slots in without a rename.

Entity	Key fields
Blueprint	<code>`id, type(dev</code>
Task	<code>`fulfiller_type(human</code>
Gate	<code>`type(approval</code>
ActuatorHandle	<code>state_backend_ref, workspace, outputs{}, status</code>
AuditEvent	<code>type, actor, task_id, correlation_id, payload, ts</code> (append-only)

4.7 EXECUTION PROFILES — DEV (LOCAL) MODE

FlowOps runs in one of two **execution profiles**, selected by config (`FLOWOPS_PROFILE=dev|cloud`). Everything *above* the actuator — the `request → task → action` model, the job state machine, the gates, the audit trail, and the entire UI — is identical across profiles. Only the substrate changes.

Concern	cloud profile	dev profile (v1)
Actuator	<code>RealActuator</code> -> OpenTofu -> AWS/GCP	<code>DummyActuator</code> — simulated <code>plan/apply/destroy</code> with realistic state transitions and timing
Execution	managed queue (SQS/Pub-Sub) + serverless container job	local DB-backed queue + in-process worker
State	S3 + DynamoDB / GCS	local (Postgres / file)
Budget gate	cloud-enforced (Budget + SCP + quota)	advisory , rendered as “dev — not enforced”
Sandbox	its own AWS account / GCP project	simulated handle (no real infra)
Teardown	delete the account/project	simulated destroy

Goal: `docker compose up` runs the complete loop — request, both gates, the full job state machine, TTL teardown, and the audit trail — on a laptop with **zero cloud credentials**. This is the contributor and out-of-the-box story, and it lets the entire product (including the lifecycle UI) be built and demoed *before* the week-0 cloud spike lands.

Invariants that still hold in dev mode:

- **Fail closed** applies in every profile (a missing cost preview still blocks approval).
- **Dev mode is always visibly dev** — a profile badge in the UI and an advisory-budget label, so a simulated budget can never be mistaken for an enforced one.
- The `DummyActuator` is the same code that serves as the test double, so dev mode and the test suite exercise one implementation.

Deliberately NOT in v1 dev mode (deferred — see “NOT in scope”): a LocalStack path that runs *real* OpenTofu against emulated AWS, and a `local-docker` blueprint whose sandbox is a usable local container. Both are natural fast-follows once the cloud loop is proven; v1 dev mode is Dummy-only by design, to stay cheap.

5. Engineering Considerations

5.1 FAILURE MODES

Failure	Covered by	Silent?
Mid-apply failure -> orphaned spend	account/project deletion + reconcile sweep + test	No — job->failed, owner alerted
Worker crash mid-apply -> double-apply	idempotency key + per-sandbox lock + redelivery test	No
Budget bypass -> runaway spend	cloud-enforced budget + SCP + quota (hard ceiling)	No — cloud kills it
Wrongful teardown -> data loss	grace period + owner notice + never-destroy guard + fail-closed	No
Missing cost/budget config	fail-closed (block apply)	No
Control-plane compromise	state encryption + secret redaction + access isolation	Residual risk — flagged, monitored

Global invariant: fail closed. Every gate and the teardown guard default to DENY on ambiguity — a missing cost estimate, an unknown role, an unparseable handle blocks, never proceeds.

5.2 TESTING STRATEGY

Full failure-path coverage. The failure tests *shape* the job model — the state machine and its failure tests are written first, before happy-path actuation. `DummyActuator` drives fast, deterministic unit and integration tests with zero cloud credentials; integration tests against real OpenTofu (localstack / a throwaway project) cover state locking and cloud-failure semantics the dummy cannot prove. Required tests include: mid-apply failure + reconcile, crash redelivery (no double-apply), budget at/over/under cap, missing-cost-preview = fail closed, teardown guardrails, and quota exhaustion.

5.3 SECURITY & SUPPLY CHAIN

- OIDC scoped-role assumption; bootstrap trust, per-account role creation, least-privilege policy, and revocation/rotation are part of the week-0 spike.
- Curated OpenTofu modules are pinned by ref, with a provider lockfile + checksums and a review policy. A “curated” module is still supply chain.
- Append-only audit with correlation IDs, actor-identity provenance, immutable job inputs, and redaction rules.

5.4 TOOLING CHOICES (BORING BY DEFAULT)

OpenTofu (not Terraform — MPL licensing fits an OSS project), Python/FastAPI control plane, Postgres, a managed queue, object-store state backend. Innovation tokens are reserved for the agent-fulfiller layer, not the plumbing.

5.5 DISTRIBUTION

AGPL-3.0 core in an OSS repo. `docker compose up` runs the **app** in minutes; the **actuation** half requires guided cloud wiring (it cannot be zero-config). CI via GitHub Actions builds/publishes container images on tag, with GitHub Releases for versioned artifacts. A hosted/managed offering is deferred until after v1 delights a self-host user.

6. Design

6.1 DESIGN SYSTEM — NYS DESIGN SYSTEM (MANDATORY)

FlowOps targets a **New York State government** context. The UI conforms to the **NYS Design System** (USWDS-derived) and is built on `---nys-color-*` design tokens (CSS custom properties) so it is themeable:

Token	Value	Use
<code>--nys-color-theme</code>	<code>#154973</code>	Primary state blue; active state, section accents
<code>--nys-color-link</code> / <code>--nys-color-focus</code>	<code>#004dd1</code>	Interactive, links, focus rings
<code>--nys-color-success</code>	<code>#1e752e</code>	Passed / healthy
<code>--nys-color-text</code>	<code>#1b1b1b</code>	Body text
<code>--nys-color-surface</code>	<code>#ffffff</code>	Page surface
Environment theme	<code>#233f2b</code>	Alternate theme

The proof-of-concept's orange admin accent is **superseded by the NYS theme blue**. Typography is an accessible government sans (Public Sans family), never `system-ui`.

Accessibility is mandatory (WCAG 2.1 AA / Section 508): visible focus rings, $\geq 4.5:1$ body contrast, 44px touch targets, full keyboard navigation, ARIA landmarks, and screen-reader labels on the status pill and lifecycle stepper. Status is never conveyed by color alone — color is always paired with icon + text.

6.2 INFORMATION ARCHITECTURE — SANDBOX LIFECYCLE VIEW

Reading order, by priority: (1) identity + current status; (2) the lifecycle stepper (where is this, right now); (3) governance (budget ceiling + Guardrails + Approve/Deny); (4) TTL countdown + Extend; (5) provisioned outputs; (6) append-only audit timeline. Status legibility is the number-one job — never plain “Status: applying” text.

6.3 APPROVED MOCKUP

The approved build reference for the sandbox lifecycle detail view (NYS Design System palette, calm layout, per-event audit iconography, guardrails panel that makes “controls never come off” literally visible):

 Approved sandbox lifecycle detail view (NYS Design System palette)

An alternate top-navigation treatment of the same screen:

 Alternate layout — persistent top nav + global search

For contrast, an earlier exploration in the original proof-of-concept palette (orange accent), before the NYS Design System constraint reshaped the direction:

 Early exploration — original PoC orange palette, bolder state machine

6.4 INTERACTION-STATE COVERAGE

Surface	Loading	Empty	Error	Success	Partial
Board card	skeleton + “Queued...”	n/a	red pill “Apply failed” + View error	green “Running” + TTL chip	amber “Applying... {step}”
Lifecycle detail	stepper skeleton	n/a	failed step red, audit shows error, Retry/Destroy	full stepper, outputs, TTL ticking	mid-stepper pulsing, outputs hidden until Running
Gate approval	“Loading cost preview...” (Approve disabled)	n/a	“Cost preview unavailable — cannot approve” (fail-closed)	Approve enabled, cost + budget shown	read-only awaiting banner for non-approvers
Failure / expiry	—	—	mid-apply failure: red banner + orphan-reconcile + owner alert	clean teardown: “Destroyed” terminal state + final audit	TTL near-expiry: amber countdown + prominent Extend

7. Strategy

Decision	Choice
License	AGPL-3.0 + commercial dual-license (requires a CLA from day one)
Open-core line	Generous core — full single-org governed-actuation loop is OSS; sell multi-tenancy, SSO/SAML/SCIM, advanced policy, hosted runners, audit retention/compliance, support
Positioning	"Governed self-service provisioning — the governed actuation layer." Sandbox = demo; AI-native ITSM = vision narrative

The AGPL + commercial structure monetizes proprietary embedding directly, which is why the feature core can be generous: the OSS version is a complete production tool for one organization, not a toy.

8. Build Plan

8.1 PARALLEL LANES

Lane	Modules	Depends on
A — control plane	<code>api/</code> , <code>models/</code> (PoC port, domain model, jobs table)	—
B — actuation	<code>actuators/</code> , jobs worker, queue wiring	A's jobs table (Dummy path early; Real path needs C)
C — cloud factory (week-0 spike)	<code>infra/</code> (account/project vending, budget/SCP, OIDC trust)	—

Launch A + C in parallel (C is the spike). B starts against `DummyActuator` immediately, integrating the Real path when C lands. A and B share the `jobs` table — one owner defines that schema first.

8.2 SEQUENCED TASKS (HIGHLIGHTS)

- Week-0 spike (blocking):** prove apply/destroy/orphan-detect + OIDC trust + budget/SCP against both GCP project and AWS account; pick the first cloud.
- Bring the PoC into the repo; module layout `api/ actuators/ gates/ jobs/ audit/ infra/`.
- Jobs/state-machine table; queue + container-job runner.
- Actuator interface + Dummy + Real; approval + cloud-enforced budget gate.
- TTL teardown via the same job path; append-only audit.
- Full failure-path test suite (written with the state machine).
- Licensing: `LICENSE` (AGPL-3.0), `COMMERCIAL.md`, CLA bot, documented open-core line.
- Design: `DESIGN.md` (NYS token mapping), build the lifecycle view from the approved mockup, implement all interaction states, WCAG 2.1 AA pass.

8.5 Security Requirements (CSO threat model, 2026-06-09)

FlowOps holds cloud credentials, executes IaC, can delete cloud accounts, and enforces spend — so these are build-blocking requirements, not hardening to add later. The plan already defends well on OIDC (no static keys), account-per-sandbox isolation, fail-closed gates, control-plane state isolation, and module pinning. The requirements below close the gaps the threat model surfaced. (AGPL = attackers can read the source; assume zero obscurity throughout.)

- SR1 [P1] — Blueprints are an admin-curated catalog; users never supply OpenTofu.** The runner executes blueprint modules with a powerful cloud role. Module source is allow-listed (pinned repo refs only); creating/editing a blueprint is an admin-gated, audited action. Prevents arbitrary cloud execution by non-admins.
- SR2 [P1] — Aggregate spend + concurrency limits (denial-of-wallet).** Per-sandbox budget is not enough. Enforce a per-user/per-org **aggregate budget**, a **rate limit** on requests, and a **cap on concurrent live sandboxes** — all fail-closed.
- SR3 [P2] — Tamper-evident audit.** Append-only is not enough; the audit *is* the product. Hash-chain audit events (each row signs the prior hash) and/or mirror to a WORM / object-lock sink so tampering is detectable. (Matters doubly for NYS compliance.)
- SR4 [P2] — Split control-plane identities.** Separate the (org-level) account-vending identity from the per-sandbox provisioning identity; use tight OIDC trust conditions (exact subject + audience, no wildcards). Resolve in the week-0 spike.

- **SR5 [P2] — No default credentials outside the `dev` profile.** The PoC's seed-on-boot default admin (`admin/admin123` etc.) must never exist in a non-dev deployment; force an admin password set on first real boot.
 - **SR6 [P2] — Sign the runner image.** SHA-pin GitHub Actions, sign images (cosign + SLSA provenance), restrict who can push release tags, and scope CI's OIDC so a compromised pipeline cannot reach prod cloud.
 - **SR7 [P2] — Explicit authz on destroy/extend.** Teardown deletes accounts; gate the destroy and TTL-extend transitions like apply — owner-or-admin only, always audited.
-

9. Open Risks

- **Control-plane blast radius:** state + secrets concentrate in the control plane. Mitigated (encryption, redaction, isolation), not eliminated — monitor.
 - **Account-factory weight:** the cloud factory could become the product before sandbox vending exists. The week-0 spike timeboxes and de-risks this.
 - **"Out of the box" honesty:** the app runs out of the box; actuation requires guided cloud setup. Positioned as a batteries-included governed loop vs a BYO toolkit — a strength, framed honestly, not a retreat.
-

Appendix A — Review Provenance

This plan was produced and pressure-tested across four structured reviews:

- **Product (office hours):** reframed work-intake PoC into AI-native ITSM / governed actuation; locked the sandbox wedge and "A + seam toward B" approach.
- **Engineering review:** locked execution model, isolation, the Actuator seam + dummy, and full failure-path testing. An independent cross-model pass (Codex, GPT-5.5) surfaced 23 challenges — integrated or routed.
- **CEO / strategy review:** locked license (AGPL + commercial), the generous open-core line, and positioning.
- **Design review:** rated the new provisioning UI 3/10 -> 8/10; surfaced the NYS Design System + accessibility constraint; approved the lifecycle mockup; specified all interaction states.

All decisions were made by the project owner; cross-model input was advisory.